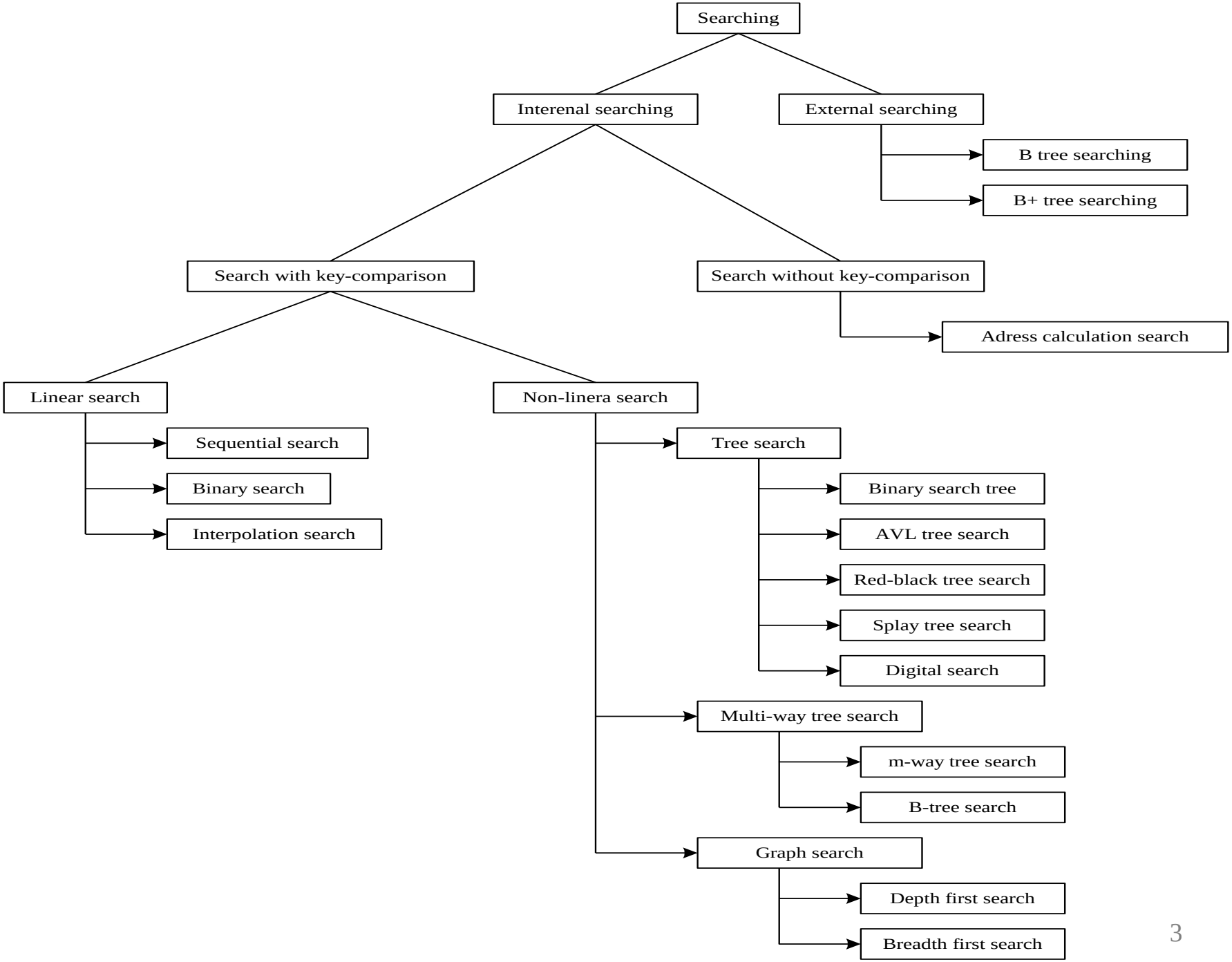
Data Structures

Searching

Searching

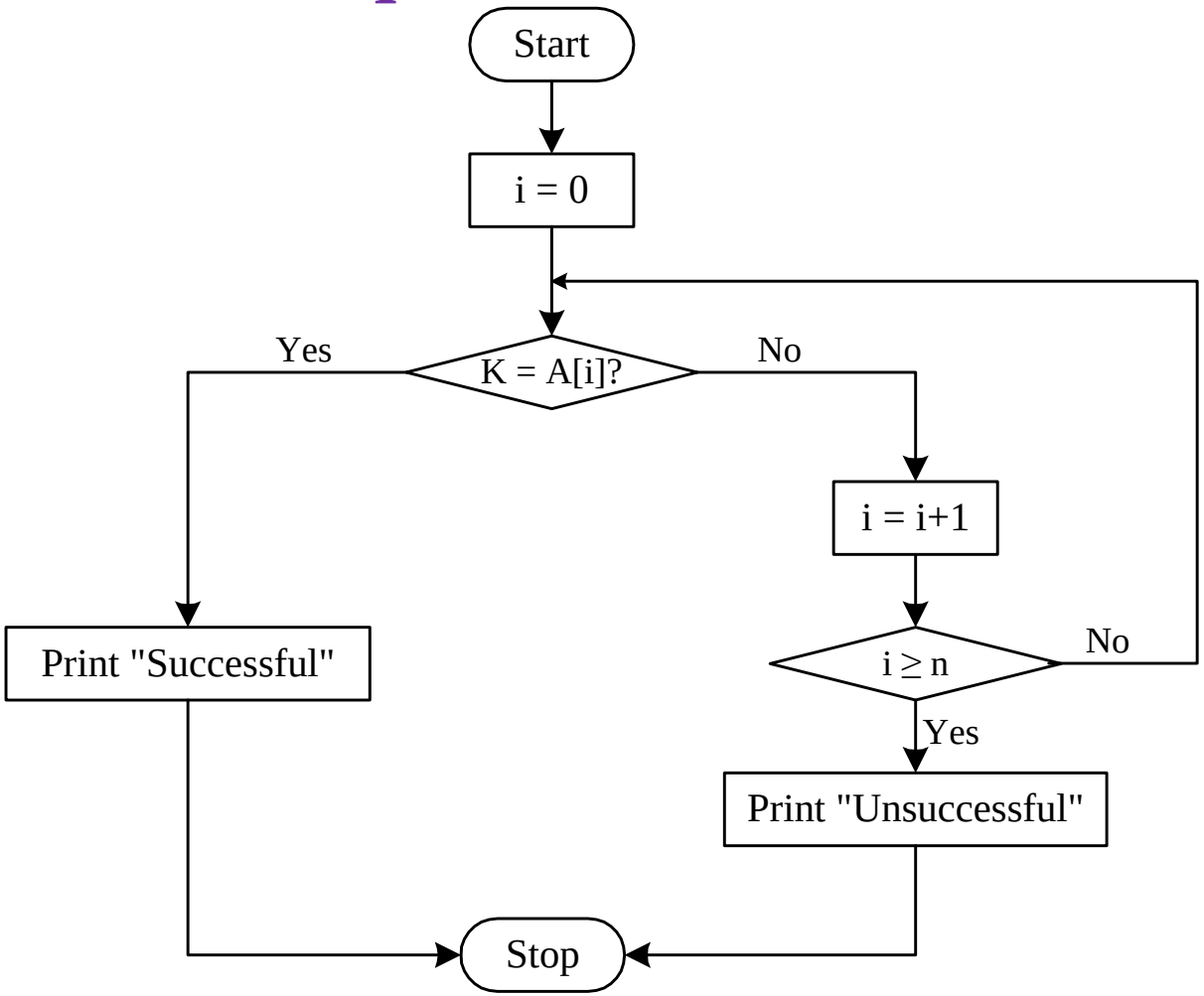
- Searching refers to the process of finding the required information from a collection of items stored as elements in the computer memory.
- These sets of items are in different forms, such as an array, linked list, graph, or tree.


Searching
Interenal searching
External searching
B tree searching
B+ tree searching
Search with key-comparison
Search without key-comparison
Adress calculation search
Linear search
Non-linera search
Sequential search
Binary search
Interpolation search
Tree search
Binary search tree
AVL tree search
Red-black tree search
Splay tree search
Digital search
Multi-way tree search
m-way tree search
B-tree search
Graph search
Depth first search
Breadth first search

Linear Search

- The linear search algorithm iteratively searches all elements of the array.
- It has the best execution time of one and the worst execution time of n, where n is the total number of items in the search array.
- It is the simplest search algorithm in data structure and checks each item in the set of elements until it matches the searched element till the end of data collection.
- When the given data is unsorted, a linear search algorithm is preferred over other search algorithms.

Flowchart: Sequential Search with Array



Example: Sequential Search with Array

```
int main()
{
    int A[10], i, n, K, flag = 0;
    printf("Enter the size of an array: ");
    scanf("%d",&n);

    printf("Enter the elements of the array: ");
    for(i=0; i < n; i++)
         scanf("%d",&A[i]);
    printf("Enter the number to be searched: ");
    scanf("%d",&K);
    for(i=0;i<n;i++){
         if(a[i] == K){
             flag = 1; break;
         }
    }
    if(flag == 0)
        printf("The number is not in the list");
    else
        printf("The number is found at index %d",i);
    return 0;
}
```

Complexity Analysis

- Case 1: The key matches with the first element
  - T(n) = 1
- Case 2: Key does not exist
  - T(n) = n
- Case 3: The key is present at any location in the array

$$T(n) = \sum_{i=1}^{n} p_i \cdot i$$

$$T(n) = \frac{1}{n}\sum_{i=1}^{n} i \qquad p_1 = p_2 = \cdots p_i = \cdots p_n = \frac{1}{n}$$

$$T(n) = \frac{n+1}{2}$$

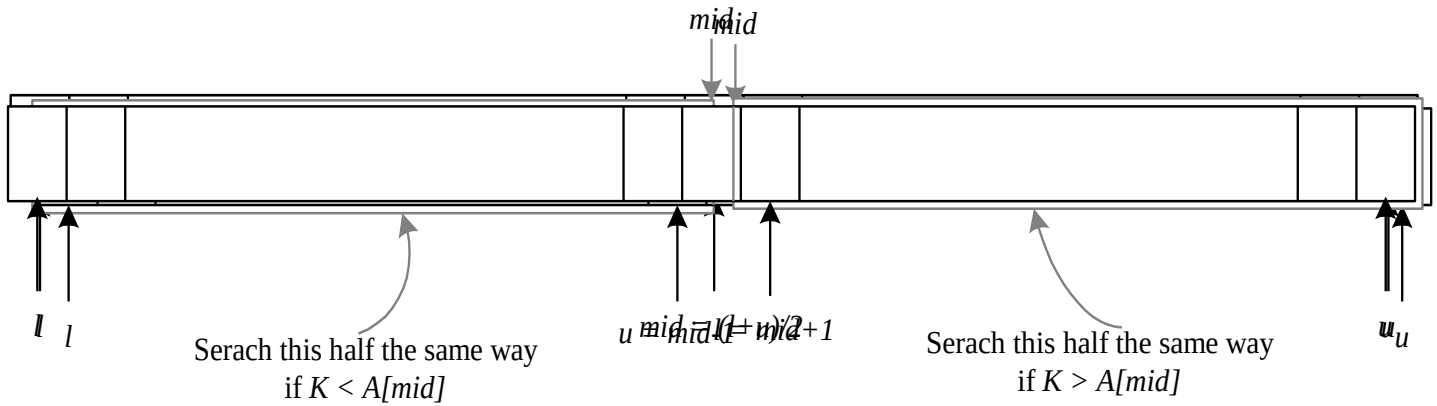
Complexity Analysis : `Summary`

Case	**Number of key comparisons**	**Asymptotic complexity**	**Remark**
Case 1	$T(n) = 1$	$T(n) = O(1)$	Best case
Case 2	$T(n) = n$	$T(n) = O(n)$	Worst case
Case 3	$T(n) = \frac{n+1}{2}$	$T(n) = O(n)$	Average case

Binary Search

- Binary search needs sorted order of items of the array.
- This algorithm looks for specific items by comparing the middlemost items in the data collection.
- When a match is found, it returns the item's index.
- When the middle item is greater than the search item, it looks for a central item of the left sub-array. If, on the other hand, the middle item is smaller than the search item, it explores the middle item in the right sub-array.
- It keeps looking for an item until it finds it or the size of the sub-arrays reaches zero.
- It works faster than a linear search algorithm. The binary search uses the divide and conquer principle.

The Technique

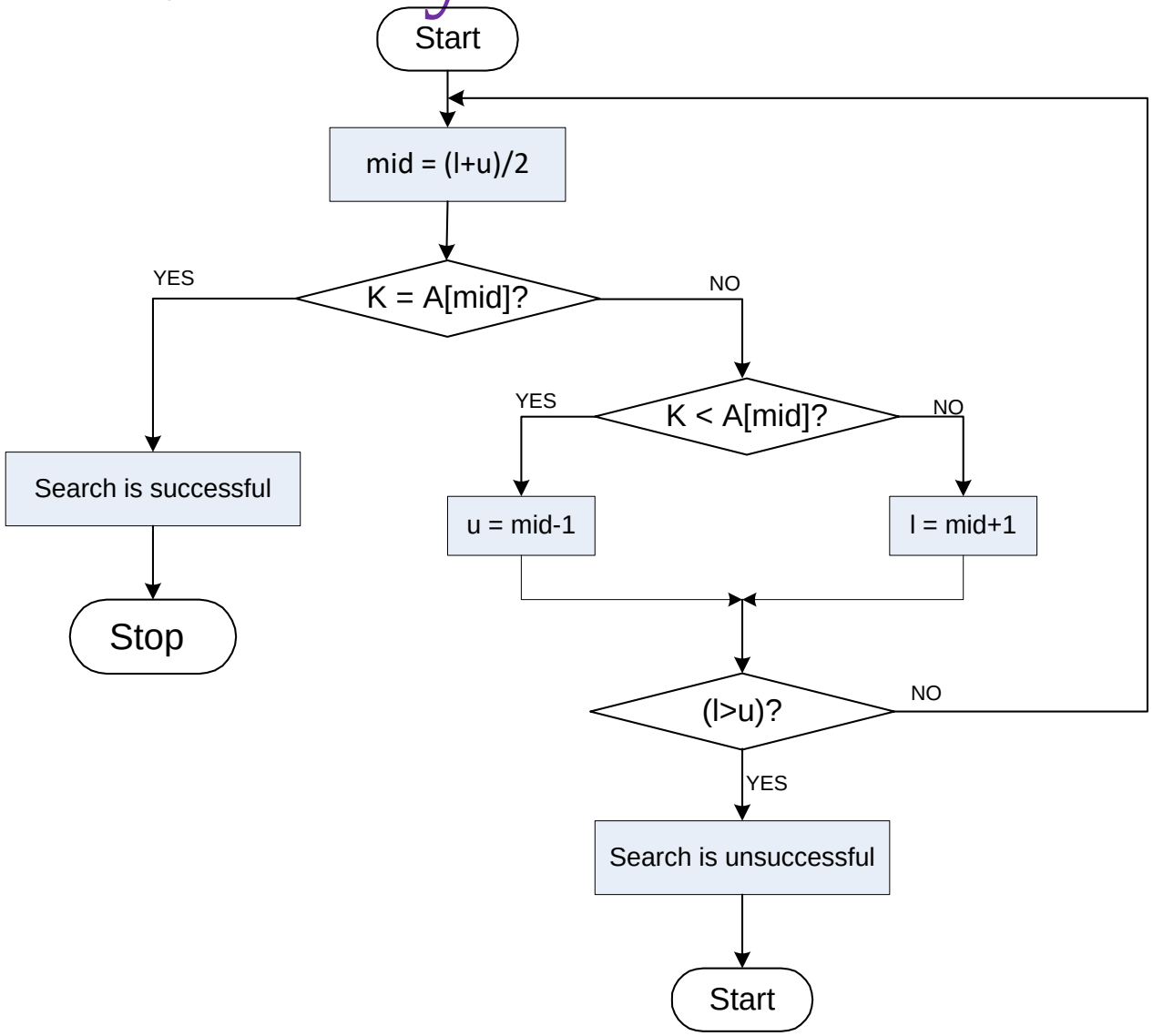


(c) Search the entire list turns into the searching of right half only

(a) At the beginning, we set the subscripts of right half to *mid*

(b) Search the entire list turns into the searching of left half only

Flowchart: Binary Search with Array



Binary Search (with Iteration)

```
#include <stdio.h>

int main()
{
   int i, l, u, mid, n, K, data[100];

   printf("Enter number of elements\n");
   scanf("%d",&n);

   printf("Enter %d integers in sorted order\n", n);

   for (i = 0; i < n; i++)
      scanf("%d",&array[i]);

   printf("Enter value to find\n");
   scanf("%d", &K);

   l = 0;
   u = n - 1;
   mid = (l+u)/2;
```

Contd…

Binary Search (with Iteration)

```
while (l <= u) {
      if (data[mid] < K)
         l = mid + 1;
      else if (data[mid] == K) {
         printf("%d found at location %d.\n", search, mid+1);
         break;
      }
      else
         u = mid - 1;

      mid = (l + u)/2;
   }
   if (l > u)
      printf("Not found! %d is not present in the list.\n", K);

   return 0;
}
```

Binary Search (with Recursion)

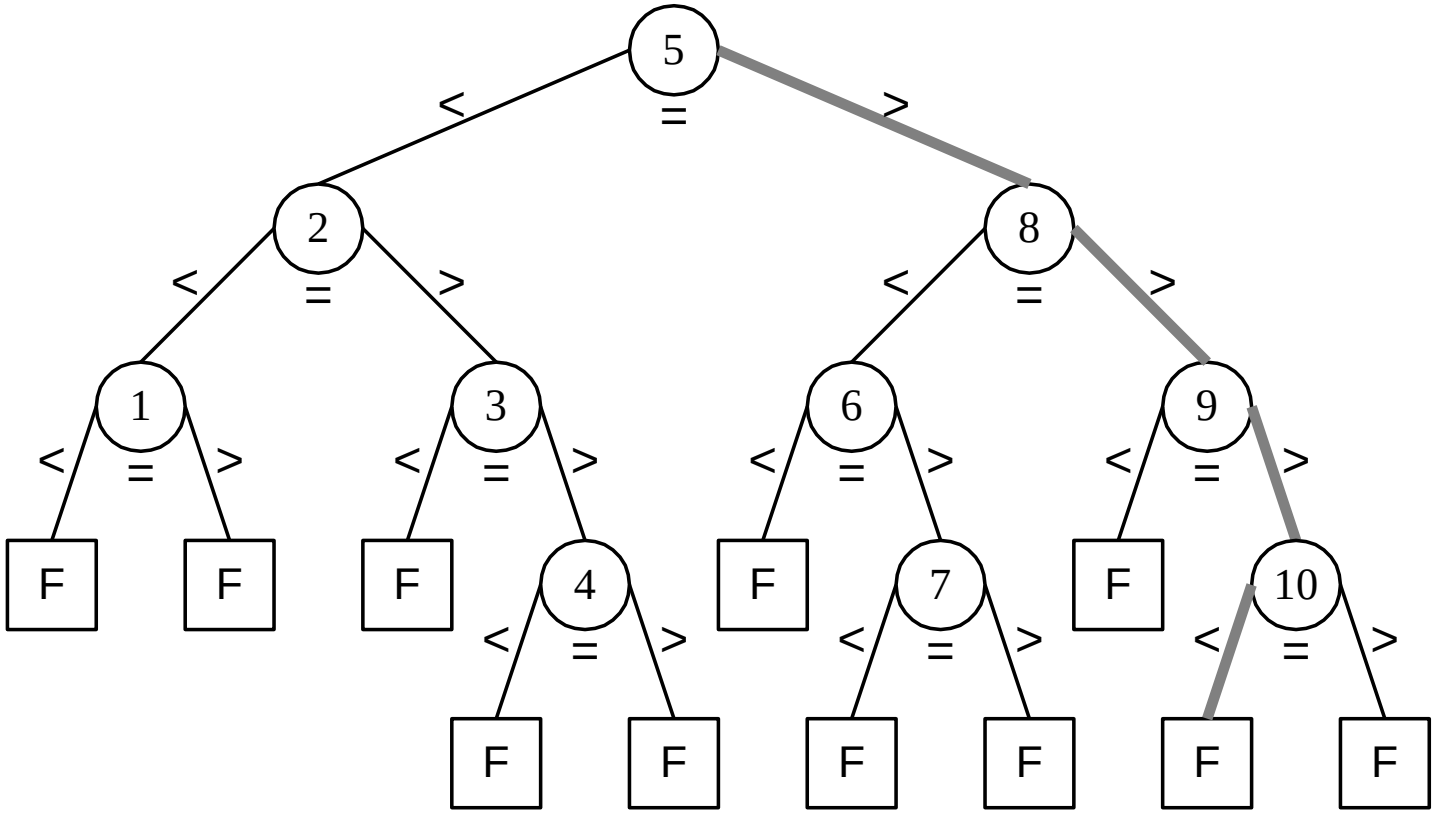
```
#include<stdio.h>
int main(){

    int data[100],i, n, K, flag, l, u;

    printf("Enter the size of an array: ");
    scanf("%d",&n);

    printf("Enter the elements of the array in sorted order: " );
    for(i=0;i<n;i++)
         scanf("%d",&a[i]);

    printf("Enter the number to be search: ");
    scanf("%d",&K);

    l=0,u=n-1;
    flag = binarySearch(data,n,K,l,u);
    if(flag==0)
         printf("Number is not found.");
    else
         printf("Number is found.");

    return 0;
 }
```

Contd…

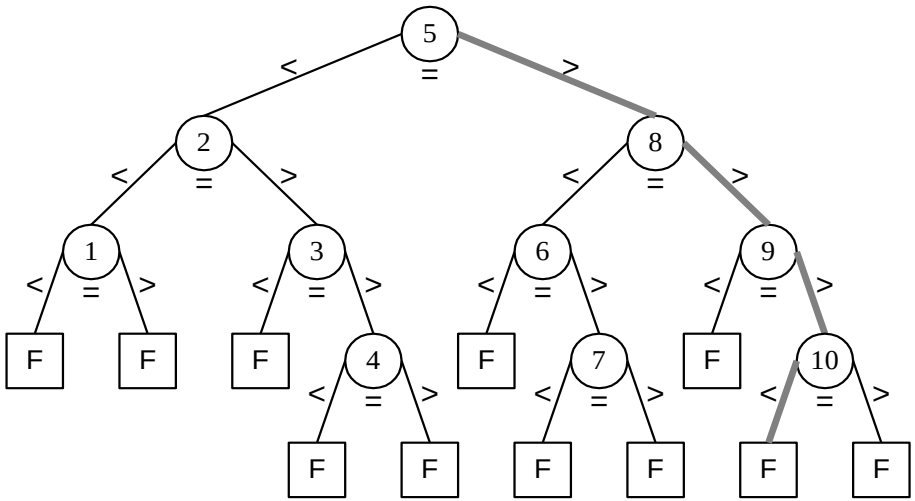
Binary Search (with Recursion)

```
int binary(int a[],int n,int K,int l,int u){

     int mid;

     if(l<=u){
          mid=(l+u)/2;
          if(K==a[mid]){
               return(1);
          }
          else if(m<a[mid]){
               return binarySearch(a,n,K,l,mid-1);
          }
          else
               return binarySearch(a,n,m,mid+1,u);
     }
     else return(0);
}
```

Complexity Analysis



Complexity Analysis: Binary Search



Let n be the total number of elements in the list under search and there exist an integer k such that:-

- For successful search:-
  - If $2^{k-1} <= n < 2^k$, then the binary search algorithm requires at least one comparison and at most k comparisons.
- For unsuccessful search:-
  - If $n = 2^{k-1}$, then the binary search algorithm requires k comparisons.
  - If $2^{k-1} <= n < 2^k-1$then the binary search algorithm requires either k-1 or k number of comparisons.

Interpolation Search

Interpolation Search

- Interpolation search is an improved variant of binary search.
- This search algorithm works on the probing position of the required value.
- For this algorithm to work properly, the data collection should be in a sorted form and equally distributed.
- Binary search has a huge advantage of time complexity over linear search. Linear search has worst-case complexity of O(n) whereas binary search has O(log n).
- There are cases where the location of target data may be known in advance. For example, in case of a telephone directory, if we want to search the telephone number of Vijay. Here, linear search and even binary search will seem slow as we can directly jump to memory space where the names start from 'V' are stored.

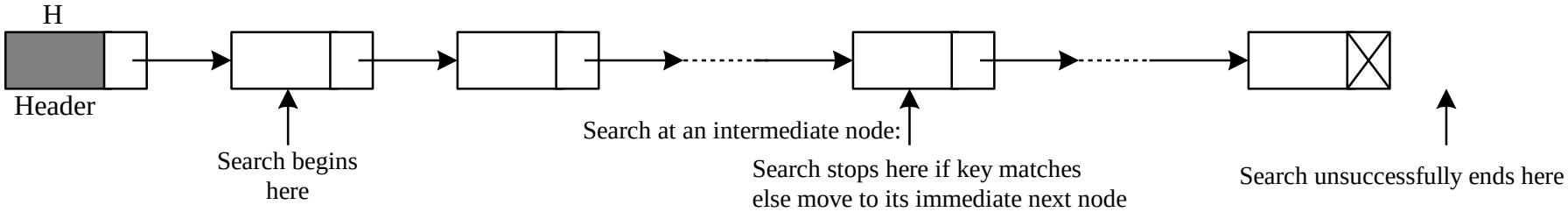
Interpolation Search

```
l = 1, u = n                                          // Initialization: Range of searching
flag = FALSE                                          // Hold the status of searching
While (flag = FALSE) do
    loc = ⌈ (K − A[l]) / (A[u] − A[l]) ⌉ × (u − l) + l
    If ( l ≤ loc ≤ u) then                            // If loc is within the range of the list
        Case:  K < A[loc]
            u = loc -1
        Case:  K = A[loc]
            flag = TRUE
        Case:  K > A[loc]
            l = loc +1
    Else
        Exit()
    EndIf
EndWhile
If (flag) then
    Print "Successful at" loc
Else
    Print "Unsuccessful"
EndIf
Stop
```

Sequential Search with Linked List

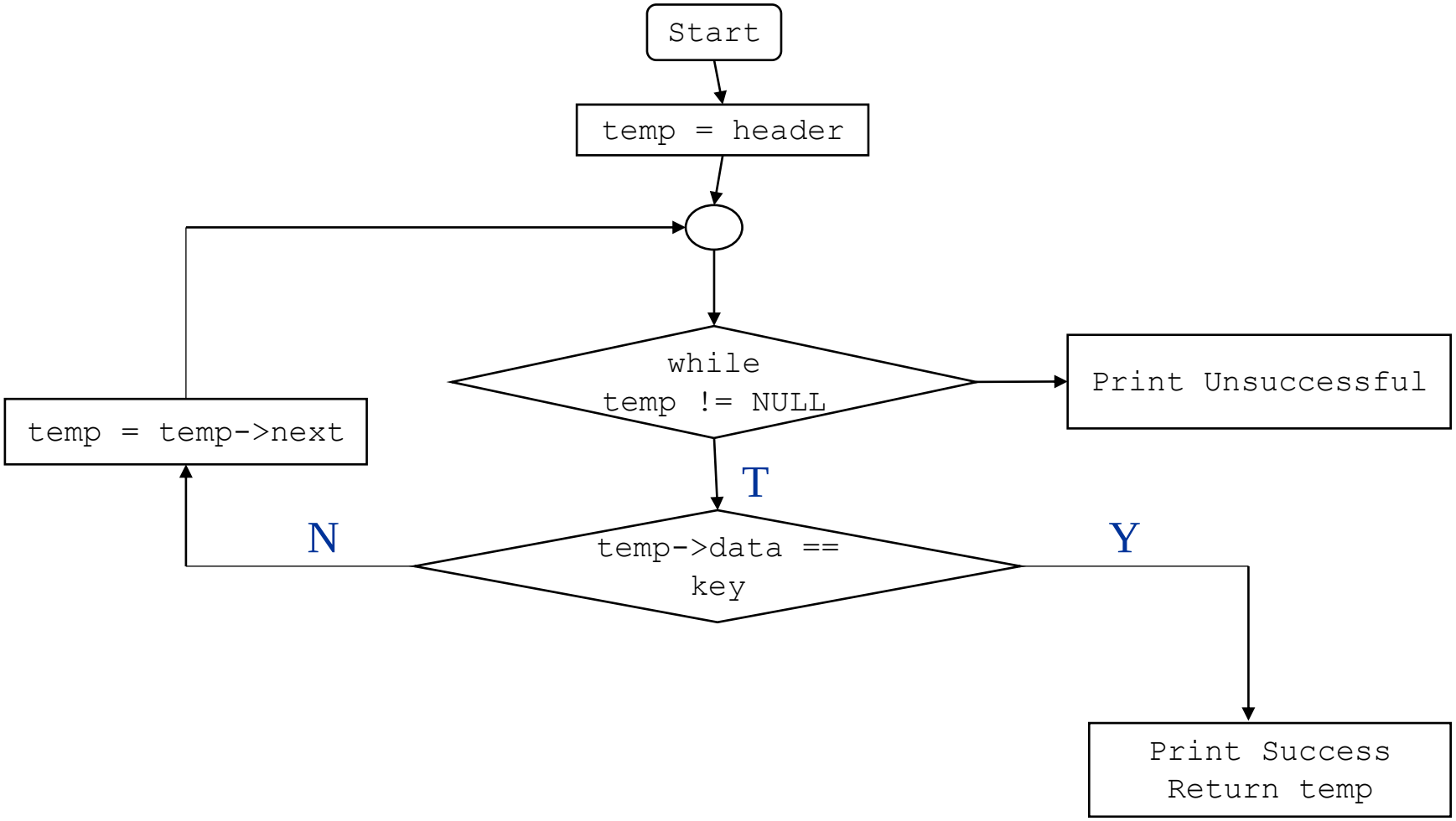
Sequential Search with Linked List

(a) Structure of a node in the linked list



(b) Linear search on a linked list

Flow Chart: Sequential Search with LL



Example: Sequential Search with LL

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    struct node *header = NULL;
    int K, n;

    printf("Enter the number of nodes: ");
    scanf("%d", &n);
    printf("\nDisplaying the list\n");
    generate(header, num);
    printf("\nEnter key to search: ");
    scanf("%d", &key);
    searchBinary(header, K);
    delete(header);

    return 0;
}
```

Example: Linear Search with LL

```
void generate(struct node *head, int n)
{
    int i;
    struct node *temp;

    for (i = 0; i < num; i++)
    {
        temp = (struct node *)malloc(sizeof(struct node));
        temp->data = rand() % n;
        if (*head == NULL)
        {
            *head = temp;
            temp->next = NULL;
        }
        else
        {
            temp->next = head;
            head = temp;
        }
        printf("%d  ", temp->data);
    }
}
```

Example: Linear Search with LL

```
void searchBinary(struct node *temp, int K)
{
    while (temp != NULL)
    {
        if (temp->data == K)
        {
            printf("key found\n");
            return;
        }
        else temp = temp->next;
    }
    printf("Key not found\n");
}

void delete(struct node *header)
{
    struct node *temp;
    temp = header;
    while (temp != NULL)
    {
        temp = temp->next;
        free(header);
        header = temp;
    }
}
```

Complexity Analysis

Case	**Number of key comparisons**	**Asymptotic complexity**	**Remark**
Case 1	$T(n) = 1$	$T(n) = O(1)$	Best case
Case 2	$T(n) = \frac{n+1}{2}$	$T(n) = O(n)$	Average case
Case 3	$T(n) = n$	$T(n) = O(n)$	Worst case

Jump Search

- Like Binary Search, Jump Search is a searching algorithm for sorted arrays.
- The basic idea is to check fewer elements (than linear search) by jumping ahead by fixed steps or skipping some elements in place of searching all elements.
- For example, suppose we have an array arr[] of size n and block (to be jumped) size m. Then we search at the indexes arr[0], arr[m], arr[2m]…..arr[km] and so on. Once we find the interval (arr[km] < x < arr[(k+1)m]), we perform a linear search operation from the index km to find the element x.

Jump Search - Example

- Let's consider the following array: (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610). Length of the array is 16. Jump search will find the value of 55 with the following steps assuming that the block size to be jumped is 4.
- STEP 1: Jump from index 0 to index 4;
- STEP 2: Jump from index 4 to index 8;
- STEP 3: Jump from index 8 to index 12;
- STEP 4: Since the element at index 12 is greater than 55 we will jump back a step to come to index 8.
- STEP 5: Perform linear search from index 8 to get the element 55.

Jump Search - Analysis

- What is the optimal block size to be skipped?
- In the worst case, we have to do n/m jumps and if the last checked value is greater than the element to be searched for, we perform m-1 comparisons more for linear search. Therefore the total number of comparisons in the worst case will be ((n/m) + m-1). The value of the function ((n/m) + m-1) will be minimum when $m = \sqrt{n}$. Therefore, the best step size is $m = \sqrt{n}$.
- So Time Complexity : $O(\sqrt{n})$

Exponential Search

- It is combination of Jump and Binary search
- Exponential search involves two steps:
  - Find range where element is present
  - Do Binary Search in above found range.
- To find the range where element may be present, start with subarray size 1, compare its last element with K, then try size 2, then 4 and so on until last element of a subarray is not greater than K.
- Once an index i is found (after repeated doubling of i), the element must be present between i/2 and i (Why i/2? because we could not find a greater value in previous iteration)
- Perform Binary search on the range i/2 to i

Exponential Search - Analysis

- It works better than Binary Search for bounded arrays, and also when the element to be searched is closer to the first element.
- Time Complexity : O(Log n)

Data Structures

Sorting

Sorting – The Task

- **Given an array**

  `x[0], x[1], … , x[size-1]`

  reorder entries so that

  `x[0] <= x[1] <=  . . . <= x[size-1]`

  Here, List is in non-decreasing order.

- **We can also sort a list of elements in non-increasing order.**

Sorting – Example

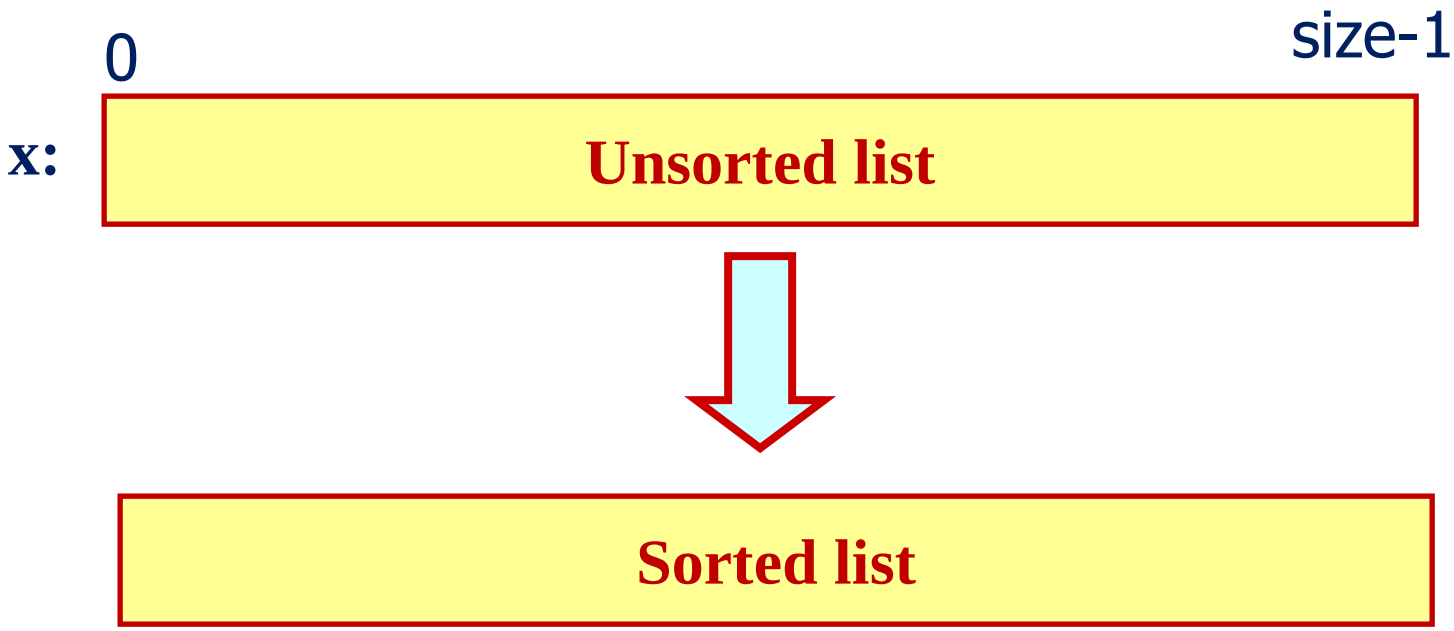
- **Original list:**
  - **10, 30, 20, 80, 70, 10, 60, 40, 70**

- **Sorted in non-decreasing order:**
  - **10, 10, 20, 30, 40, 60, 70, 70, 80**

- **Sorted in non-increasing order:**
  - **80, 70, 70, 60, 40, 30, 20, 10, 10**

Sorting Problem

- **What do we want :**
  - **Data to be sorted in order**



Issues in Sorting

Many issues are there in sorting techniques

- How to rearrange a given set of data?
- Which data structures are more suitable to store data prior to their sorting?
- How fast the sorting can be achieved?
- How sorting can be done in a memory constraint situation?
- How to sort various types of data?

Sorting Algorithms

Sorting by Comparison

- Basic operation involved in this type of sorting technique is comparison. A data item is compared with other items in the list of items in order to find its place in the sorted list.
  - Insertion
  - Selection
  - Exchange
  - Enumeration

Sorting by Comparison

Sorting by comparison – Insertion:

- From a given list of items, one item is considered at a time. The item chosen is then inserted into an appropriate position relative to the previously sorted items. The item can be inserted into the same list or to a different list.

e.g.: Insertion sort

Sorting by comparison – Selection:

- First the smallest (or largest) item is located and it is separated from the rest; then the next smallest (or next largest) is selected and so on until all item are separated.

e.g.: Selection sort, Heap sort

Sorting by Comparison

Sorting by comparison – Exchange:

- If two items are found to be out of order, they are interchanged. The process is repeated until no more exchange is required.

  e.g.: Bubble sort, Shell Sort, Quick Sort

Sorting by comparison – Enumeration:

- Two or more input lists are merged into an output list and while merging the items, an input list is chosen following the required sorting order.

  e.g.: Merge sort

Sorting by Distribution

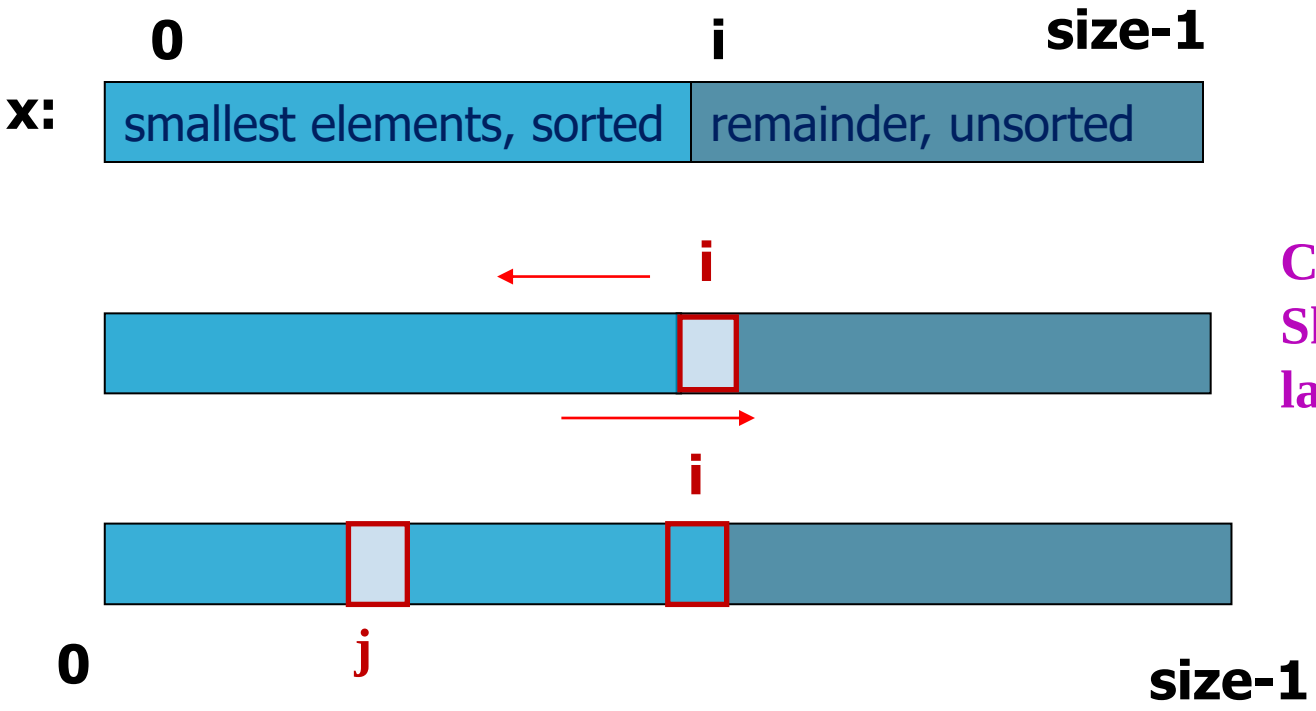
- No key comparison takes place
- All items under sorting are distributed over an auxiliary storage space based on the constituent element in each and then grouped them together to get the sorted list.
- Distributions of items based on the following choices
  - ✓ **Radix -** An item is placed in a space decided by the bases (or radix) of its components with which it is composed of.
  - ✓ **Counting -** Items are sorted based on their relative counts.
  - ✓ **Hashing** - Items are hashed, that is, dispersed into a list based on a hash function.

Note: This lecture concentrates only on sorting by comparison.

Insertion Sort

Insertion Sort

General situation :



Compare and Shift till x[i] is larger.

Insertion Sort

```
void insertionSort (int list[], int size)
{
        int i,j,item;

        for (i=1; i<size; i++)
        {
                item = list[i] ;
/* Move elements of list[0..i-1], that are greater than
item, to one position ahead of their current position */

                for (j=i-1; (j>=0)&& (list[j] > item); j--)
                        list[j+1] = list[j];
                list[j+1] = item ;
        }
}
```

Insertion Sort

```
int main()
{
    int x[ ]={-45,89,-65,87,0,3,-23,19,56,21,76,-50};

    int i;
    for(i=0;i<12;i++)
       printf("%d ",x[i]);
    printf("\n");

    insertionSort(x,12);

    for(i=0;i<12;i++)
       printf("%d ",x[i]);
    printf("\n");
}
```

OUTPUT

-45 89 -65 87 0 3 -23 19 56 21 76 -50

-65 -50 -45 -23 0 3 19 21 56 76 87 89

Insertion Sort - Example

54	26	93	17	77	31	44	55	20	Assume 54 is a sorted list of 1 item
26	54	93	17	77	31	44	55	20	inserted 26
26	54	93	17	77	31	44	55	20	inserted 93
17	26	54	93	77	31	44	55	20	inserted 17
17	26	54	77	93	31	44	55	20	inserted 77
17	26	31	54	77	93	44	55	20	inserted 31
17	26	31	44	54	77	93	55	20	inserted 44
17	26	31	44	54	55	77	93	20	inserted 55
17	20	26	31	44	54	55	77	93	inserted 20

Insertion Sort: Complexity Analysis

Case 1: If the input list is already in sorted order

Number of comparisons: Number of comparison in each iteration is 1.

$$C(n) = 1 + 1 + 1 + \cdots + 1 \quad \text{upto (n-1)}^{\text{th}} \text{ iteration.}$$

Number of movement: No data movement takes place in any iteration.

$$M(n) = 0$$

Insertion Sort: Complexity analysis

Case 2: If the input list is sorted but in reverse order

Number of comparisons: Number of comparison in each iteration is 1.

$$C(n) = 1 + 2 + 3 + \cdots + (n-1) = \frac{n(n-1)}{2}$$

Number of movement: Number of movements takes place in any i^{th} iteration is i.

$$M(n) = 1 + 2 + 3 + \cdots + (n-1) = \frac{n(n-1)}{2}$$

Insertion Sort: Complexity analysis

Case 3: If the input list is in random order

- Let p_j be the probability that the key will go to the jth location $(1 \leq j \leq i+1)$. Then the number of comparisons will be $j \cdot p_j$.

- The average number of comparisons in the $(\mathrm{i}+1)^{th}$ iteration is

$$A_{i+1} = \sum_{j=1}^{i+1} j \cdot p_j$$

- Assume that all keys are distinct and all permutations of keys are equally likely.

$$p_1 = p_2 = p_3 = \cdots = p_{i+1} = \frac{1}{i+1}$$

Insertion Sort: Complexity analysis

Case 3: Number of comparisons

- Therefore, the average number of comparisons in the $(i+1)^{th}$ iteration

$$A_{i+1} = \frac{1}{i+1}\sum_{j=1}^{i+1} j = \frac{1}{i+1}\cdot\frac{(i+1)\cdot(i+2)}{2} = \frac{i+2}{2}$$

- Total number of comparisons for all $(n-1)$ iterations is

$$C(n) = \sum_{i=0}^{n-1} A_{i+1} = \frac{1}{2}\cdot\frac{n(n-1)}{2} + (n-1)$$

Insertion Sort: Complexity analysis

Case 3: Number of Movements

- On the average, number of movements in the ith iteration

$$M_i = \frac{i + (i-1) + (i-2) + \cdots + 2 + 1}{i} = \frac{i+1}{2}$$

- Total number of movements

$$M(n) = \sum_{i=1}^{n-1} M_i = \frac{1}{2} \cdot \frac{n(n-1)}{2} + \frac{n-1}{2}$$

Insertion Sort: Summary of Complexity Analysis

Case	Comparisons	Movement	Memory	Remarks
Case 1	$C(n) = (n-1)$	$M(n) = 0$	$S(n) = n$	Input list is in sorted order
Case 2	$C(n) = \frac{n(n-1)}{2}$	$M(n) = \frac{n(n-1)}{2}$	$S(n) = n$	Input list is sorted in reverse order
Case 3	$C(n) = \frac{(n-1)(n+4)}{4}$	$M(n) = \frac{(n-1)(n+2)}{4}$	$S(n) = n$	Input list is in random order

Case	Run time, $T(n)$	Complexity	Remarks
Case 1	$T(n) = c\,(n-1)$	$T(n) = O(n)$	Best case
Case 2	$T(n) = c\,n(n-1)$	$T(n) = O(n^2)$	Worst case
Case 3	$T(n) = c\,\frac{(n-1)(n+3)}{2}$	$T(n) = O(n^2)$	Average case

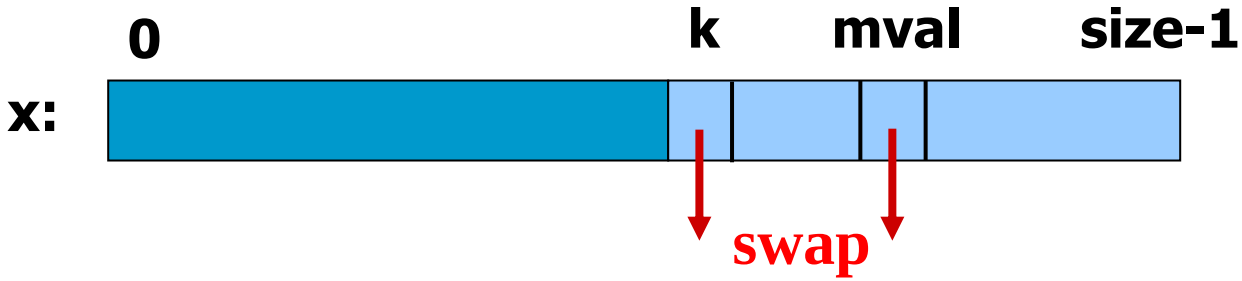
Selection Sort

Selection Sort

General situation :

Steps :

- **Find smallest element, mval, in x[k…size-1]**
- **Swap smallest element with x[k], then increase k.**



Selection Sort

```
/* Yield location of smallest element in
x[k .. size-1];*/

int findMinLloc (int x[ ], int k, int size)
{
    int j, pos;       /* x[pos] is the smallest
element found so far */
      pos = k;
      for (j=k+1; j<size; j++)
            if (x[j] < x[pos])
                  pos = j;
      return pos;
}
```

Selection Sort

```
/* The main sorting function */
/* Sort x[0..size-1] in non-decreasing order */

int selectionSort (int x[], int size)
{   int k, m;
      for (k=0; k<size-1; k++)
     {
            m = findMinLoc(x, k, size);
            temp = a[k];
            a[k] = a[m];
            a[m] = temp;
      }
}
```

Selection Sort - Example

x:	3	12	-5	6	142	21	-17	45
x:	-17	12	-5	6	142	21	3	45
x:	-17	-5	12	6	142	21	3	45
x:	-17	-5	3	6	142	21	12	45
x:	-17	-5	3	6	142	21	12	45
x:	-17	-5	3	6	12	21	142	45

x:	-17	-5	3	6	12	21	142	45
x:	-17	-5	3	6	12	21	45	142
x:	-17	-5	3	6	12	21	45	142

Selection Sort: Complexity Analysis

Case 1: If the input list is already in sorted order

Number of comparisons:

$$C(n) = \sum_{i=1}^{n-1} (n - i) = \frac{n(n-1)}{2}$$

Number of movement: no data movement takes place in any iteration.

$$M(n) = 0$$

Selection Sort: Complexity Analysis

Case 2: If the input list is sorted but in reverse order

Number of comparisons:

$$c(n) = \sum_{i=1}^{n-1} (n - i) = \frac{n(n-1)}{2}$$

Number of movements:

$$M(n) = \frac{3}{2}(n - 1)$$

Selection Sort: Complexity Analysis

Case 3: If the input list is in random order

Number of comparisons:

$$C(n) = \frac{n(n-1)}{2}$$

- Let p_i be the probability that the ith smallest element is in the ith position. Number of total swap operations $= (1 - p_i) \times (n - 1)$

  where $p_1 = p_2 = p_3 = \cdots = p_n = \frac{1}{n}$

- **Total number of movements**

$$M(n) = \left(1 - \frac{1}{n}\right) \times (n-1) \times 3 = \frac{3(n-1)(n-1)}{n}$$

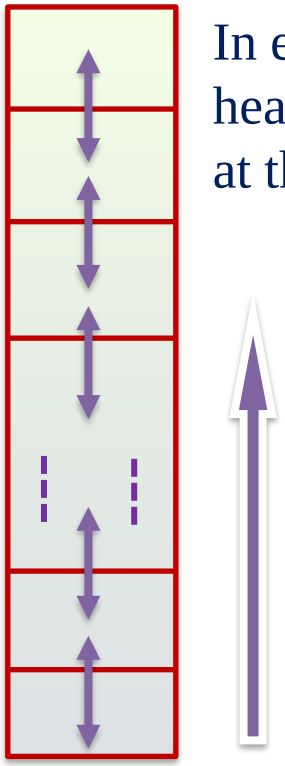
Selection Sort: Summary of Complexity analysis

Case	Comparisons	Movement	Memory	Remarks
Case 1	$c(n) = \frac{n(n-1)}{2}$	$M(n) = 0$	$S(n) = 0$	Input list is in sorted order
Case 2	$c(n) = \frac{n(n-1)}{2}$	$M(n) = \frac{3(n-1)}{2}$	$S(n) = 0$	Input list is sorted in reverse order
Case 3	$c(n) = \frac{n(n-1)}{2}$	$M(n) = \frac{3(n-1)^2}{n}$	$S(n) = 0$	Input list is in random order

Case	Run time, $T(n)$	Complexity	Remarks
Case 1	$T(n) = \frac{n(n-1)}{2}$	$T(n) = O(n^2)$	Best case
Case 2	$T(n) = \frac{(n-1)(n+3)}{2}$	$T(n) = O(n^2)$	Worst case
Case 3	$T(n) \approx \frac{(n-1)(2n+3)}{2}$ (Taking $n-1 \approx n$)	$T(n) = O(n^2)$	Average case

Bubble Sort

Bubble Sort



In every iteration heaviest element drops at the bottom.

The bottom moves upward.

The sorting process proceeds in several passes.

- In every pass we go on comparing neighbouring pairs, and swap them if out of order.
- In every pass, the largest of the elements under considering will *bubble* to the top (i.e., the right).

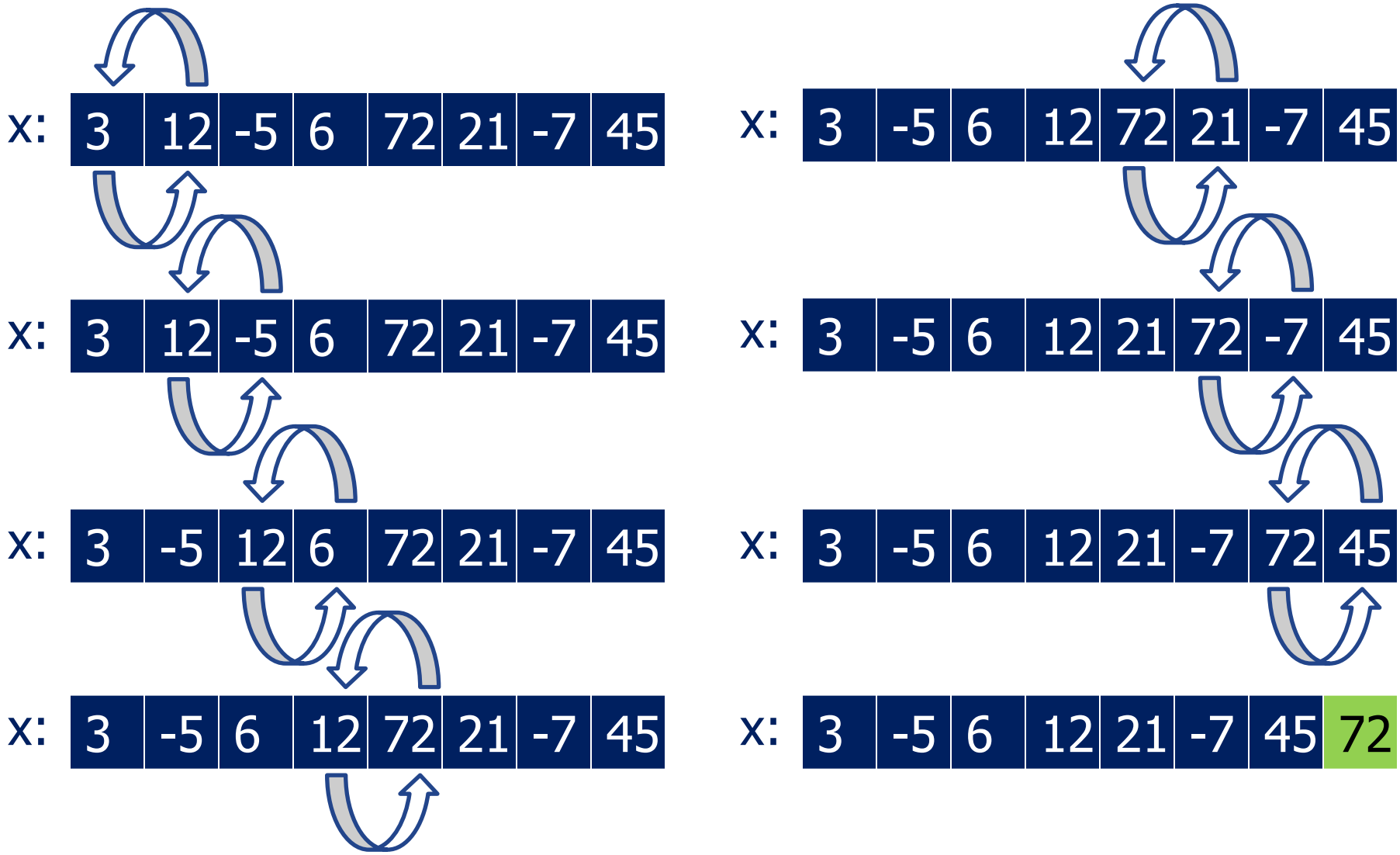
Bubble Sort

How the passes proceed?

- In **pass 1**, we consider index **0 to n-1.**
- In **pass 2**, we consider index **0 to n-2.**
- In **pass 3**, we consider index **0 to n-3.**
- ……
- ……
- In **pass n-1**, we consider index **0 to 1.**

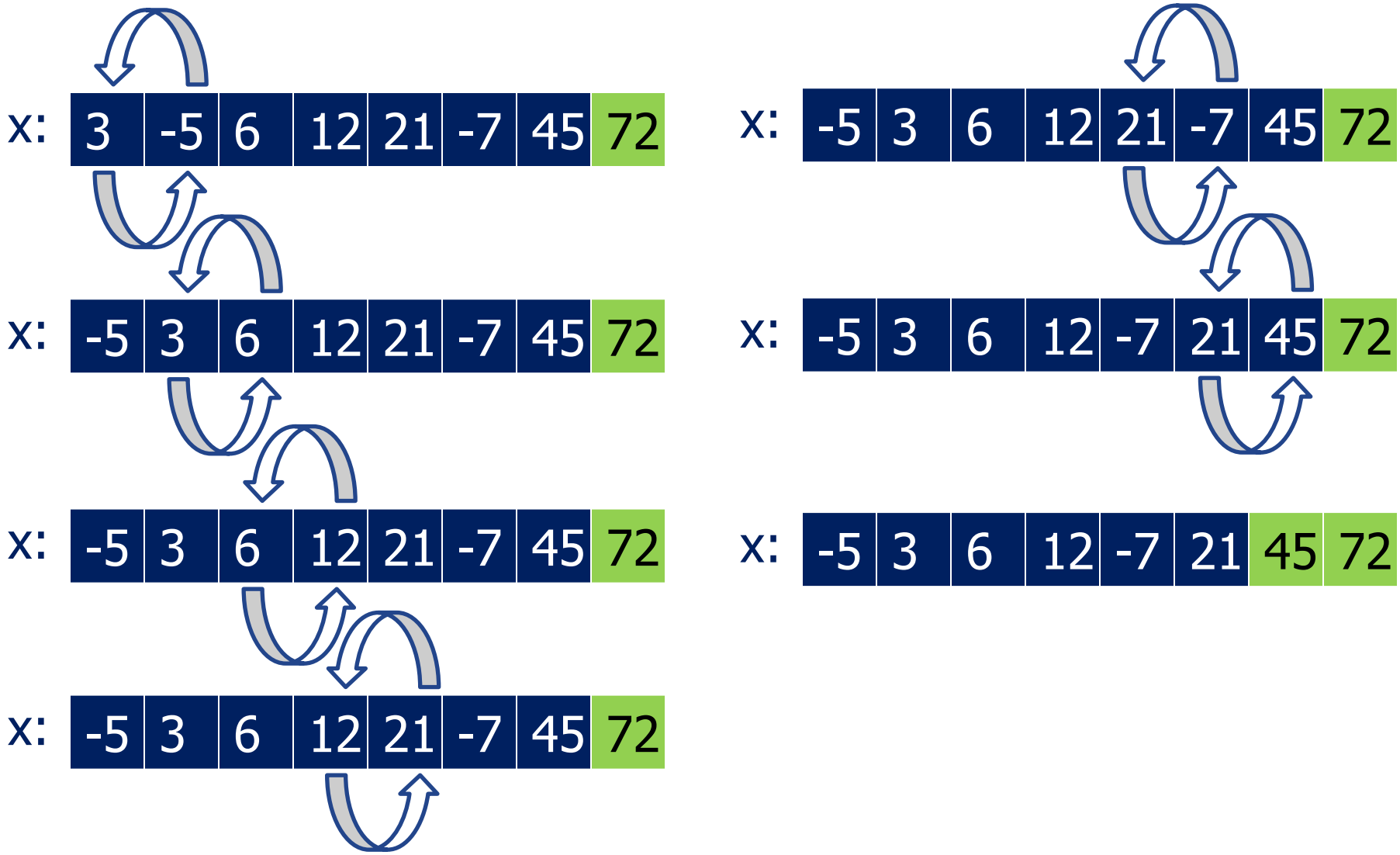
Bubble Sort - Example

Pass: 1



Bubble Sort - Example

Pass: 2



Bubble Sort

```
void swap(int *x, int *y)
{
      int tmp = *x;
      *x = *y;
      *y = tmp;
}

void bubble_sort(int x[], int n)
{
      int i,j;
      for (i=n-1; i>0; i--)
         for (j=0; j<i; j++)
            if (x[j] > x[j+1])
               swap(&x[j],&x[j+1]);
}
```

Bubble Sort

```
int main()
{
  int x[ ]={-45,89,-65,87,0,3,-23,19,56,21,76,-50};
  int i;
  for(i=0;i<12;i++)
     printf("%d ",x[i]);
  printf("\n");
  bubble_sort(x,12);
  for(i=0;i<12;i++)
     printf("%d ",x[i]);
  printf("\n");
}
```

OUTPUT

-45 89 -65 87 0 3 -23 19 56 21 76 -50

-65 -50 -45 -23 0 3 19 21 56 76 87 89

Bubble Sort: Complexity analysis

Case 1: If the input list is already in sorted order

Number of comparisons:

$$C(n) = \frac{n(n-1)}{2}$$

Number of movements:

$$M(n) = 0$$

Bubble Sort: Complexity analysis

Case 2: If the input list is sorted but in reverse order

Number of comparisons:

$$c(n) = \frac{n(n-1)}{2}$$

Number of movements:

$$M(n) = \frac{n(n-1)}{2}$$

Bubble Sort: Complexity analysis

Case 3: If the input list is in random order

Number of comparisons:

$$C(n) = \frac{n(n-1)}{2}$$

Number of movements:

- Let p_j be the probability that the largest element is in the unsorted part is in j^{th} $(1 \leq j \leq n - i + 1)$ location.
- The average number of swaps in the ith pass is

$$= \sum_{j=1}^{n-i+1} (\overline{n - i + 1} - j) \cdot p_j$$

Bubble Sort: Complexity analysis

Case 3: If the input list is in random order

Number of movements:

- $p_1 = p_2 = p_{n-i+1} = \frac{1}{n-i+1}$
- Therefore, the average number of swaps in the ith pass is

$$= \sum_{j=1}^{n-i+1} \frac{1}{n-i+1} \cdot (\overline{n-i+1} - j) = \frac{n-i}{2}$$

- The average number of movements

$$M(n) = \sum_{i=1}^{n-1} \frac{n-i}{2} = \frac{n(n-1)}{4}$$

Bubble Sort: Summary of Complexity analysis

Case	Comparisons	Movement	Memory	Remarks
Case 1	$c(n) = \frac{n(n-1)}{2}$	$M(n) = 0$	$S(n) = 0$	Input list is in sorted order
Case 2	$c(n) = \frac{n(n-1)}{2}$	$M(n) = \frac{n(n-1)}{2}$	$S(n) = 0$	Input list is sorted in reverse order
Case 3	$c(n) = \frac{n(n-1)}{2}$	$M(n) = \frac{n(n-1)}{4}$	$S(n) = 0$	Input list is in random order

Case	Run time, $T(n)$	Complexity	Remarks
Case 1	$T(n) = c\frac{n(n-1)}{2}$	$T(n) = O(n^2)$	Best case
Case 2	$T(n) = cn(n-1)$	$T(n) = O(n^2)$	Worst case
Case 3	$T(n) = \frac{3}{4}n(n-1)$	$T(n) = O(n^2)$	Average case

Bubble Sort

How do you make best case with (n-1) comparisons only?

- By maintaining a variable flag, to check if there has been any swaps in a given pass.
- If not, the array is already sorted.

Bubble Sort

```
void bubble_sort(int x[], int n)
{
  int i,j;
  int flag = 0;
   for (i=n-1; i>0; i--)
   {
     for (j=0; j<i; j++)
     if (x[j] > x[j+1])
     {
       swap(&x[j],&x[j+1]);
       flag = 1;
     }
     if (flag == 0) return;
   }
}
```

Efficient Sorting algorithms

Two of the most popular sorting algorithms are based on divide-and-conquer approach.

- Quick sort
- Merge sort

Basic concept of divide-and-conquer method:

```
sort (list)
{
    if the list has length greater than 1
    {
        Partition the list into lowlist and highlist;
        sort (lowlist);
        sort (highlist);
        combine (lowlist, highlist);
    }
}
```

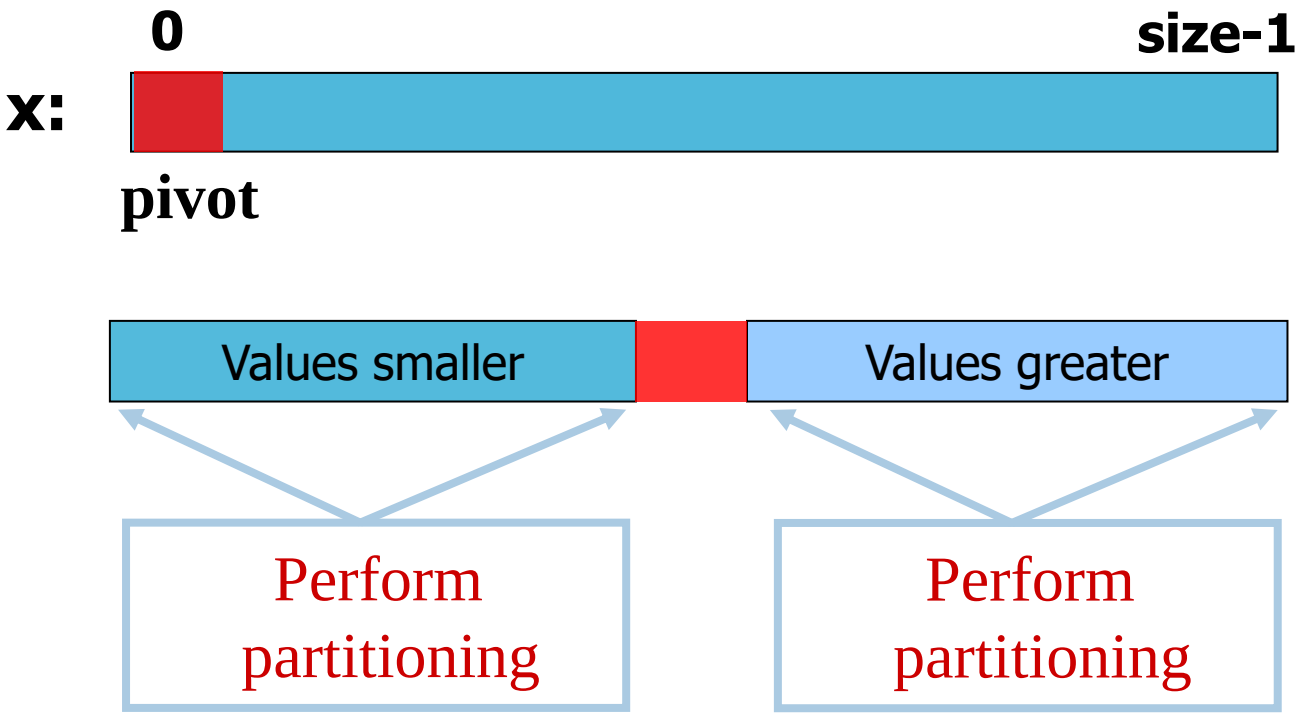
Quick Sort

Quick Sort – How it Works?

At every step, we select a *pivot element* in the list (usually the first element).

- We put the pivot element in the *final position* of the sorted list.
- All the elements less than or equal to the pivot element are to the left.
- All the elements greater than the pivot element are to the right.

Quick Sort Partitioning



Quick Sort

```
#include <stdio.h>
void quickSort( int[], int, int);
int partition( int[], int, int);
void main()
{
   int i,a[] = { 7, 12, 1, -2, 0, 15, 4, 11, 9};
   printf("\n\nUnsorted array is: ");
   for(i = 0; i < 9; ++i)
      printf(" %d ", a[i]);
   quickSort( a, 0, 8);
   printf("\n\nSorted array is: ");
   for(i = 0; i < 9; ++i)
      printf(" %d ", a[i]);
}
void quickSort( int a[], int l, int r)
{
   int j;
   if( l < r ) {  // divide and conquer
   j = partition( a, l, r);
   quickSort( a, l, j-1);
   quickSort( a, j+1, r);
   }
}
```

Quick Sort

```
int partition( int a[], int l, int r)
{
   int pivot, i, j, t;
   pivot = a[l];
   i = l;
   j = r+1;
   while( 1) {
          do {
              ++i;
             } while(a[i]<=pivot && i<=r);
          do {
              --j;
             } while( a[j] > pivot );
          if( i >= j ) break;
          t = a[i];
          a[i] = a[j];
          a[j] = t;
   }
   t = a[l];
   a[l] = a[j];
   a[j] = t;
   return j;
}
```

Quick Sort - Example

Input: 45 -56 78 90 -3 -6 123 0 -3 45 69 68

45 -56 78 90 -3 -6 123 0 -3 45 69 68

-6 -56 -3 0 -3 45 123 90 78 45 69 68

-56 -6 -3 0 -3 68 90 78 45 69 123

-3 0 -3 45 68 78 90 69

-3 0 69 78 90

Output: -56 -6 -3 -3 0 45 45 68 69 78 90 123

Quick Sort: Complexity analysis

Memory requirement:

Size of the stack is:

$$S(n) = \lceil log_2 n \rceil + 1$$

Number of comparisons:

- Let, $T(n)$ represents total time to sort n elements and $P(n)$ represents the time for perform a partition of a list of n elements.

$$T(n) = P(n) + T(n_l) + T(n_r), \quad \text{with } T(1) = T(0) = 0$$

where, n_l = number of elements in the left sub list

n_r = number of elements in the right sub list and $0 \leq n_l, n_r < n$

Quick Sort: Complexity analysis

Case 1: Elements in the list are in ascending order

Number of comparisons:

$$C(n) = n - 1 + C(n-1), \text{ with } C(1) = C(0) = 0$$

$$C(n) = (n-1) + (n-2) + (n-3) + \cdots + 2 + 1$$

$$C(n) = \frac{n(n-1)}{2}$$

Number of movements:

$$M(n) = 0$$

Quick Sort: Complexity analysis

Case 2: Elements in the list are in reverse order

Number of comparisons:

$$C(n) = n - 1 + C(n-1), \text{ with } C(1) = C(0) = 0$$

$$C(n) = (n-1) + (n-2) + (n-3) + \cdots + 2 + 1$$

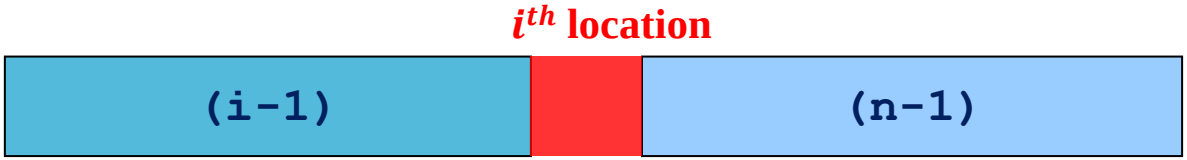
$$C(n) = \frac{n(n-1)}{2}$$

Number of movements:

$$M(n) = \begin{cases} \frac{n-1}{2}, if\ n\ is\ odd \\ \frac{n}{2}, if\ n\ is\ even \end{cases}$$

Quick Sort: Complexity analysis

Case 3: Elements in the list are in random order



Number of comparisons:

$$C(n) = (n-1) + \sum_{i=1}^{n-1} \frac{1}{n}[C(i-1) + C(n-1)] \quad with\ C(1) = C(0) = 0$$

$$C(n) = (n-1) + \frac{2}{n}\sum_{i=1}^{n-1} C(i)$$

$$C(n) = 2(n+1)(log_e n + 0.577) - 4n$$

Quick Sort: Complexity analysis

Case 3: Elements in the list are in random order

Number of movements:

$$M(n) = \frac{1}{n}\sum_{i=1}^{n}[(i-1) + M(i-1) + M(n-i)]$$

$$M(n) = \frac{n-1}{2} + \frac{2}{n}\sum_{i=1}^{n-1} M(i)$$

$$M(n) = 2(n+1)(log_e n + 0.577) - 4n$$

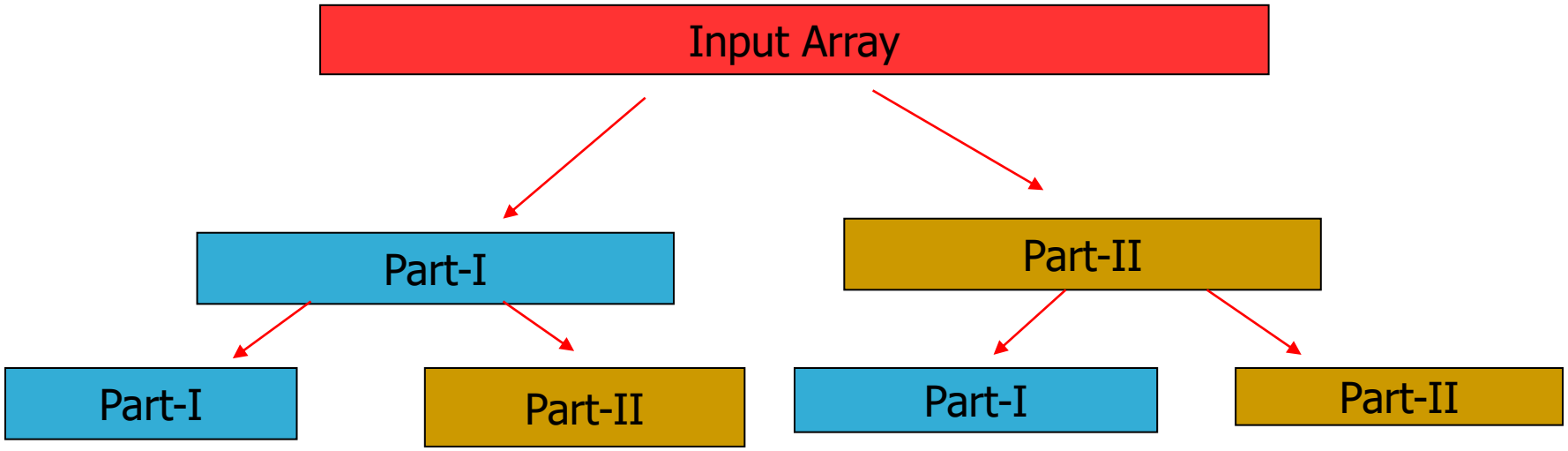
Quick Sort: Summary of Complexity analysis

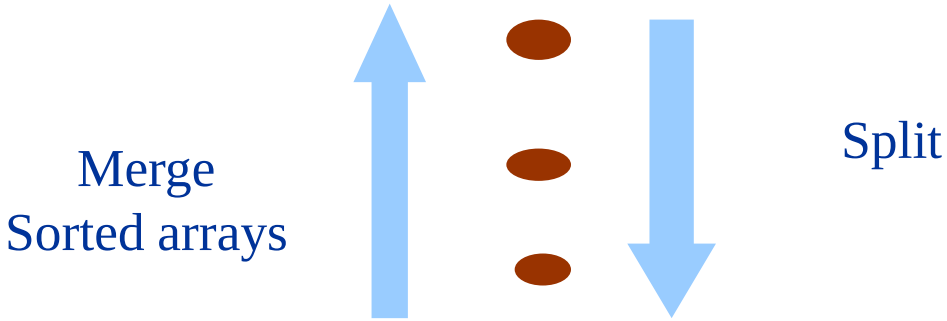
Case	Comparisons	Movement	Memory	Remarks
Case 1	$c(n) = \frac{n(n-1)}{2}$	$M(n) = 0$	$S(n) = 1$	Input list is in sorted order
Case 2	$c(n) = \frac{n(n-1)}{2}$	$M(n) = \left\lfloor \frac{n}{2} \right\rfloor$	$S(n) = 1$	Input list is sorted in reverse order
Case 3	$C(n) = 2(n+1)(log_e n + 0.577) - 4n$	$M(n) = 2(n+1)(log_e n + 0.577) - 4n$	$S(n) = \lceil log_2 n \rceil + 1$	Input list is in random order

Case	Run time, $T(n)$	Complexity	Remarks
Case 1	$T(n) = c\frac{n(n-1)}{2}$	$T(n) = O(n^2)$	Worst case
Case 2	$T(n) = c\left(\frac{n(n-1)}{2} + \left\lfloor \frac{n}{2} \right\rfloor\right)$	$T(n) = O(n^2)$	Worst case
Case 3	$T(n) = 4c(n+1)(log_e n + 0.577) - 8cn$ $T(n) = 2c[nlog_2 n - n + 1]$	$T(n) = O(nlog_2 n)$	Best / Average case

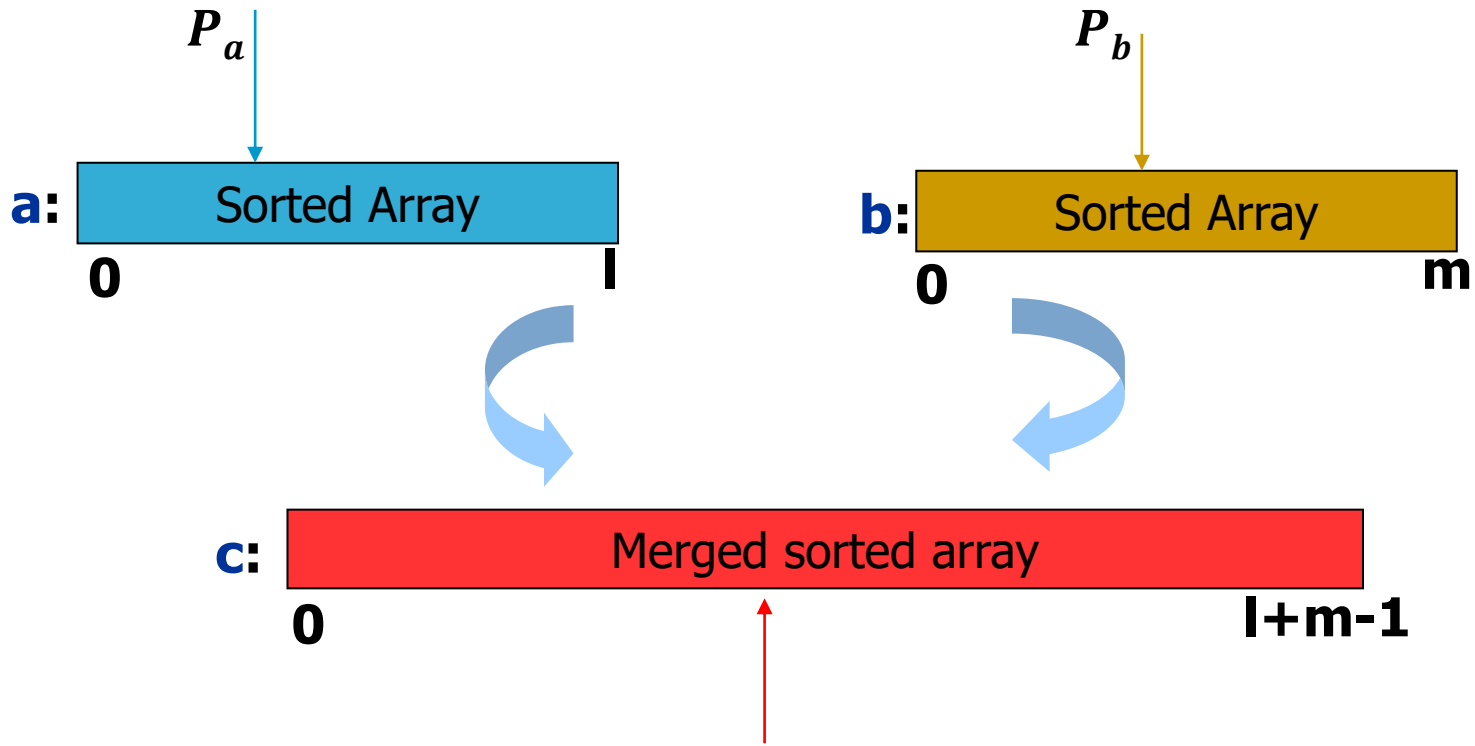
Merge Sort

Merge Sort – How it Works?



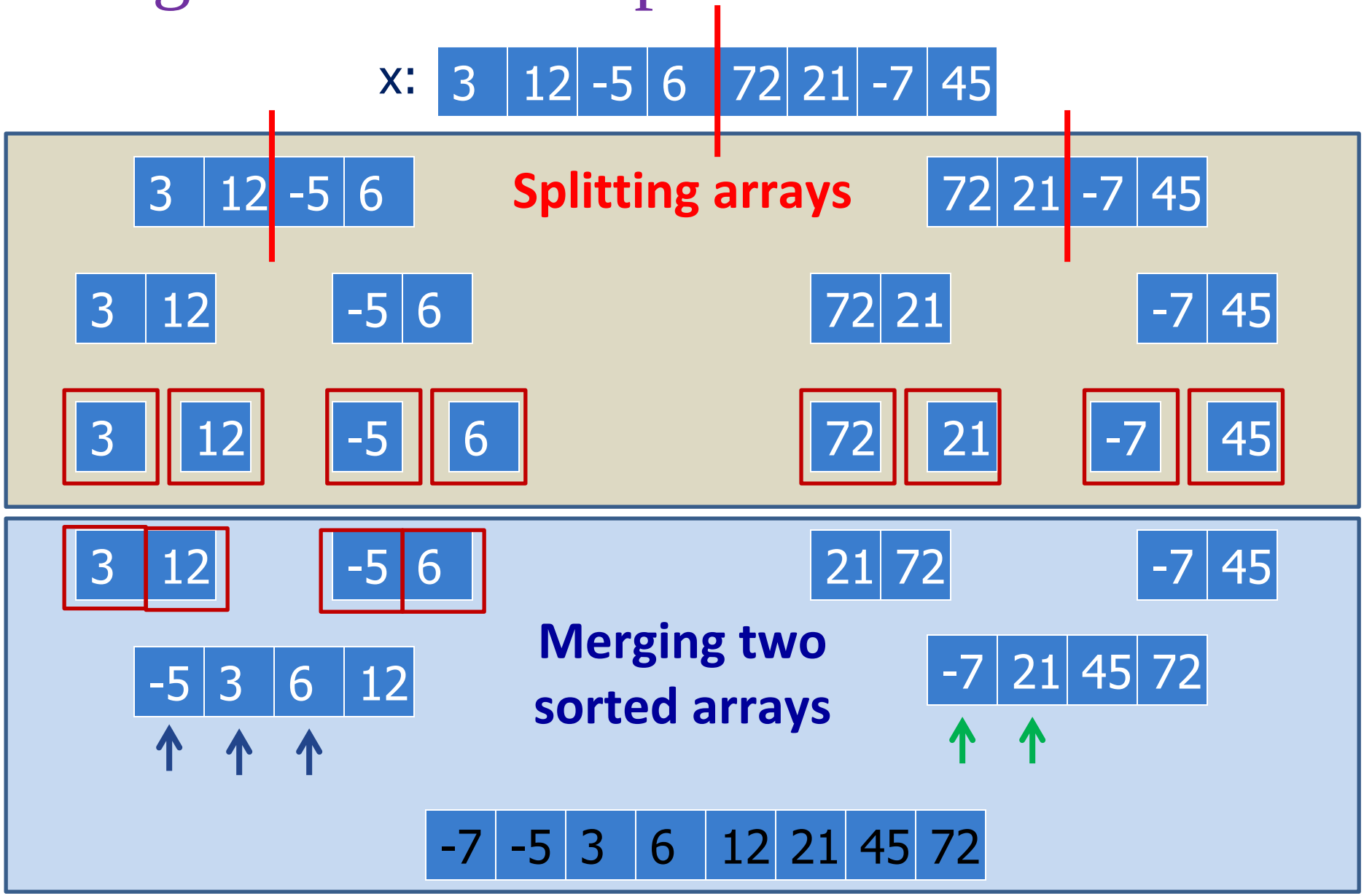


Merging two Sorted arrays



Move and copy elements pointed by P_a if its value is smaller than the element pointed by P_b in $(l + m - 1)$ operations and otherwise.

Merge Sort – Example



Merge Sort Program

```
#include<stdio.h>
void mergesort(int a[],int i,int j);
void merge(int a[],int i1,int j1,int i2,int j2);

int main()
{
    int a[30],n,i;
    printf("Enter no of elements:");
    scanf("%d",&n);
    printf("Enter array elements:");

    for(i=0;i<n;i++)
      scanf("%d",&a[i]);

    mergesort(a,0,n-1);

    printf("\nSorted array is :");
    for(i=0;i<n;i++)
      printf("%d ",a[i]);

    return 0;
}
```

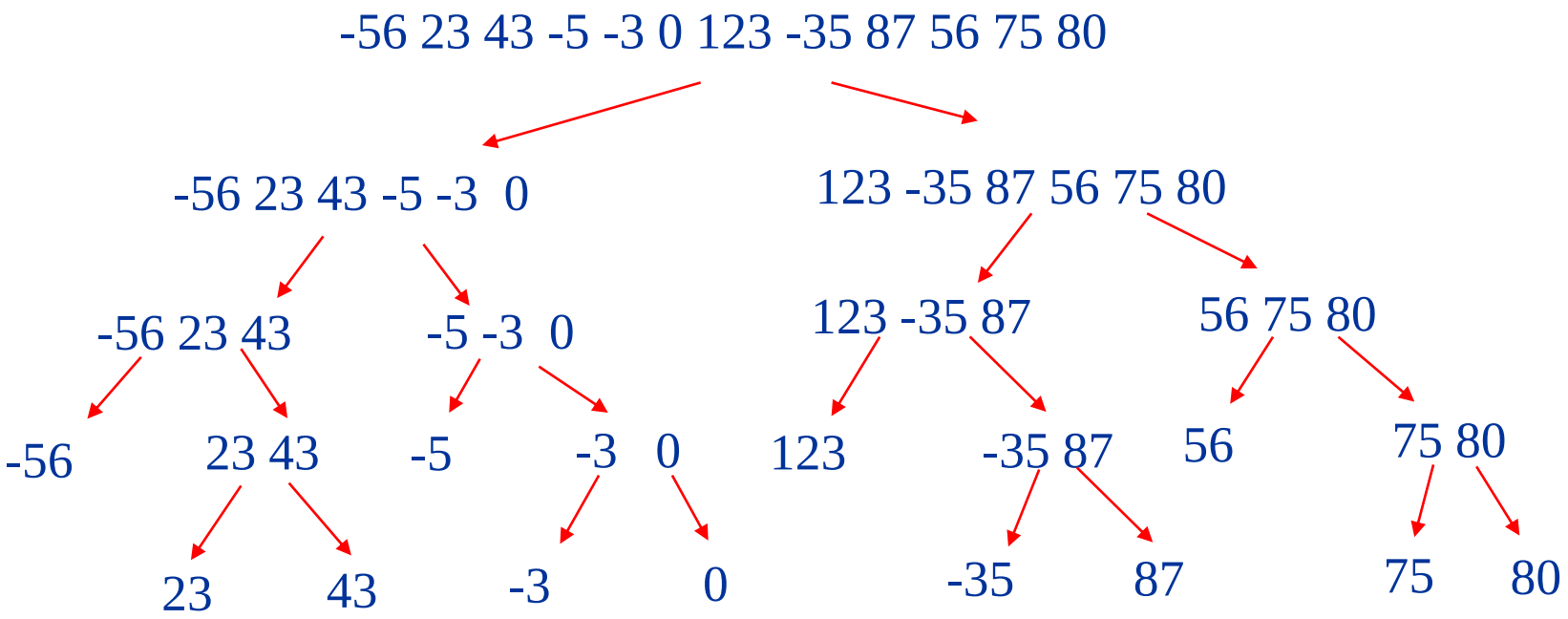
Merge Sort Program

```
void mergesort(int a[],int i,int j)
{
    int mid;

    if(i<j) {
    mid=(i+j)/2;
   /* left recursion */
    mergesort(a,i,mid);
   /* right recursion */
    mergesort(a,mid+1,j);
 /* merging of two sorted sub-arrays */
    merge(a,i,mid,mid+1,j);
    }
}
```

Merge Sort Program

```
void merge(int a[],int i1,int i2,int j1,int j2)
{
    int temp[50]; //array used for merging
    int i=i1,j=j1,k=0;

    while(i<=i2 && j<=j2) //while elements in both lists
    {
      if(a[i]<a[j])
         temp[k++]=a[i++];
      else
         temp[k++]=a[j++];
     }

    while(i<=i2) //copy remaining elements of the first list
       temp[k++]=a[i++];

    while(j<=j2) //copy remaining elements of the second list
       temp[k++]=a[j++];

    for(i=i1,j=0;i<=j2;i++,j++)
    a[i]=temp[j]; //Transfer elements from temp[] back to a[]
}
```

Merge Sort – Splitting Trace



Output: -56 -35 -5 -3 0 23 43 56 75 80 87 123

Space Complexity??

Worst Case: O(n.log(n))

Merge Sort: Complexity analysis

Time Complexity:

$$T(n) = D(n) + T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + C(n) \qquad if\ n > 1$$

$$T(n) = c_1 \qquad if\ n = 1$$

- For simplicity of calculation

$$T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) = T\left(\left\lceil \frac{n}{2} \right\rceil\right) = T\left(\frac{n}{2}\right)$$

$$T(n) = c_1 \qquad if\ n = 1$$

$$T(n) = c_2 + 2T\left(\frac{n}{2}\right) + (n-1) \qquad if\ n > 1$$

$$T(n) = n \cdot c_1 + nlog_2 n + (c_2 - 1)(n-1) \qquad Assuming\ n = 2^k$$

Quick Sort vs. Merge Sort

- **Quick sort**
  - **hard division, easy combination**
  - partition in the divide step of the divide-and-conquer framework
  - hence combine step does nothing
- **Merge sort**
  - **easy division, hard combination**
  - merge in the combine step
  - the divide step in this framework does one simple calculation only

Quick Sort vs. Merge Sort

Both the algorithms divide the problem into two sub problems.

- **Merge sort:**
  - – two sub problems are of almost equal size always.
- **Quick sort:**
  - – an equal sub division is not guaranteed.
- This difference between the two sorting methods appears as the deciding factor of their run time performances.